



TECHNICAL OVERVIEW — FULL PROJECT HISTORY & PLAYBOOK

GPS Ushering and Events — How It Was Built, and How to Build the Next One

A complete, chronological account of every decision, pivot, bug and fix from the first intake questionnaire to the current production deployment — written as a personal reference for this project and a reusable playbook for future client builds.

1. Executive Summary — Where This Came From, Where It Is Now

GPS Ushering and Events is a Ghana-based ushering and event-support business. The brief started as a standard business-website request — seven pages, a black/gold/burgundy/white brand identity — and grew, over the course of the build, into a full custom content-management platform with its own admin panel, because the client needed to be able to run the site herself without any git or GitHub literacy.

The single biggest decision of the project was abandoning the initial Node.js/Eleventy static-site approach entirely, partway through, in favor of a pure Python stack (FastAPI + Jinja2 + SQLAlchemy). That pivot was not a technical failure — Eleventy worked fine — it was a fluency decision: the developer maintaining this project long-term is Python-fluent and a JavaScript newcomer, and a stack that's unfamiliar to its own maintainer is a liability regardless of how popular or "correct" it is. Every architectural choice after that point (content-as-database-rows instead of Markdown files, a database-backed admin panel instead of a git-based CMS, Leaflet instead of the Google Maps API) followed the same underlying principle: optimize for the people who actually have to operate this thing day to day — a Python-only developer and a non-technical business owner — not for what's trendy.

THEN (PROJECT START)	NOW (CURRENT STATE)
Node.js + Eleventy, static Markdown content, Decap CMS planned for editing	Pure Python: FastAPI + Jinja2 + SQLAlchemy, zero Node/npm/build step
No admin panel — content lived in git-committed files	One unified, database-backed admin panel with full CRUD over every content type and bookings
Static Google Maps embed on the Book Us page	Interactive Leaflet + OpenStreetMap picker — click or drag a pin to set the event location
SQLite only, running locally	Postgres in production (Railway plugin), SQLite still used for local dev only
Deployed nowhere	Live on Railway at a temporary *.up.railway.app URL, fully functional, database-backed, waiting only on the client's domain decision
No documentation for anyone else	A client-facing project overview, two client-facing pricing briefs (blank and fully priced), and this document

What's left before this goes fully live under the client's own domain: the client has not yet decided on a domain name. Everything else — the code, the database, the admin panel, the hosting — is ready and tested. Once a domain is chosen, the remaining steps are purely infrastructural (DNS + one environment variable) and are covered in Section 9.

2. Full Chronological History

Phase 1 — Initial Build (Node.js / Eleventy)

Client intake questionnaire → first working site

Work began from a detailed client intake questionnaire describing GPS Ushering and Events' services, brand colors (black, gold, burgundy, white), and the seven pages needed (Home, About, Services, Gallery, Testimonials, FAQ, Book Us). The first working version was built as a static site with Eleventy (11ty), with content stored as Markdown files with YAML front matter, following a fairly conventional static-site-generator pattern. Git commits were made at every completed stage per the developer's instruction ("commit every stage as you complete them"), and the repository was linked to GitHub (Kofi-Astro/Ushering) early on.

Several Eleventy-specific issues surfaced immediately: the build crawled into `node_modules` and a Python `virtualenv` folder by default (fixed with `.eleventyignore`); a YAML settings data file silently failed to load because Eleventy doesn't parse YAML data files without extra configuration (fixed by converting it to JSON); content-collection Markdown files were rendering as unwanted standalone pages (fixed with `per-directory permalink: false` control files); and Eleventy's default pretty-URL output broke existing links (fixed with explicit `permalink: front matter` on every page). All of this was later rendered moot by Phase 2, but it's recorded here because the underlying lesson — a chosen tool's default behavior is rarely exactly what you want, and each of these required active discovery, not assumption — is a general one.

Phase 2 — The Stack Pivot: Node/Eleventy → Python/FastAPI

The single largest architectural decision in the project

Once the site was functional, the developer flagged that they were "totally a newbie to javascript" and asked to move the entire stack onto something Python-based — specifically FastAPI, run via `uvicorn` — since that's what they're actually fluent in. This was treated as a full rewrite, not a wrapper: Eleventy, `package.json`, `node_modules`, and every piece of Node tooling were deleted outright. The new stack became FastAPI for routing, Jinja2 (via Starlette's `Jinja2Templates`) for server-rendered HTML templates with real template inheritance, and `uvicorn` as the ASGI server — with zero build step of any kind. The developer explicitly accepted that some client-side JavaScript would remain unavoidable for browser-only interactivity (mobile nav toggle, theme switch, FAQ accordion, gallery lightbox) once it was explained that this is presentation-layer glue rather than application logic, and doesn't conflict with "the stack is Python."

Phase 3 — Adding an Admin Panel, Then Unifying It

Decap CMS considered and explicitly rejected

The developer asked for a way for the business owner to self-manage content and bookings without developer involvement. The first design used SQLite/SQLAlchemy for bookings, plus a separate git-commit-based CMS (Decap CMS, requiring a GitHub OAuth app) for editing the Markdown content files. This was reconsidered and reversed: the developer had never used Decap CMS before, and more importantly wanted the business owner to have full self-service control without needing any git/GitHub literacy at all — a git-commit-based CMS fundamentally requires understanding commits, branches, and a GitHub account, which defeats the purpose for a non-technical end user. Decap CMS and the OAuth router were deleted entirely. All content (services, testimonials, gallery items, FAQs, site-wide settings) was re-modeled as SQLAlchemy tables — the same database that already held bookings — and a full CRUD admin interface was built for each content type. Content became "content-as-database-rows," edited through ordinary HTML forms, with no concept of git exposed to the end user anywhere.

Phase 4 — Admin Panel on Its Own Subdomain

Host()-based virtual hosting instead of a path prefix

The developer decided the admin panel should live on `admin.<domain-name>` rather than under a path like `/admin`. This was implemented using Starlette's `Host()` route matcher: the admin panel is built as a completely separate FastAPI app (`app.admin.admin_app`) with its own clean top-level routes (`/`, `/login`, `/bookings`, `/services`, ...), and it is grafted onto the main app's router, matched against the incoming request's `Host` header. This avoids the common headache of path-prefix mounting, where every internal link and redirect needs prefix-rewriting logic — because the admin app is reached via a different hostname entirely, every one of its own top-level paths just works, unmodified, regardless of which hostname the request arrived on.

Phase 5 — Feature Additions: Interactive Map, Broader SEO

The Book Us page's static Google Maps embed was replaced with an interactive picker: Leaflet.js rendering OpenStreetMap tiles, with a draggable marker that reverse-geocodes the picked coordinates via OpenStreetMap's free Nominatim API and writes the resulting address into the location field. This was a deliberate choice over the Google Maps JavaScript API specifically to avoid requiring the client to set up and maintain a billed Google Cloud project just for a map widget — Leaflet + OSM + Nominatim needs no API key and no billing account at all. Separately, SEO keyword targeting was broadened across every page to cover the full breadth of terms relating to ushering, events, and staffing services in Ghana, embedded both in visible page copy and in a JSON-LD LocalBusiness schema block on the homepage.

Phase 6 — Documentation Pass

Three categories of documentation were produced, each for a different audience, deliberately kept as separate documents rather than one combined file: a **project overview** for the client (ideal timeline, what the client needs to prepare before development begins), **two variants of a client-facing pricing brief** — one with pricing left blank except for domain/hosting costs, one fully priced at researched top-of-market Ghana rates — so the developer could choose which to send without a back-and-forth, and this **technical overview** for the developer's own future reference. The pricing brief went through several iterations at the developer's request: real pricing, then blank pricing, then finally both variants side by side; and its voice was corrected from plural ("we") to first-person singular ("I"), since this is a solo developer's business, not a team's.

Phase 7 — Full Code Commenting Pass

Every file in the codebase was annotated with comments explaining intent and non-obvious behavior, specifically so the developer could pick the project back up after a gap without needing to re-derive how any part of it works. This is reflected throughout the current codebase — see Section 5 for the file-by-file structure.

Phase 8 — Admin Panel Visual Redesign

The admin panel was functionally complete but visually generic — plain dashboard styling that didn't reflect the brand at all. The developer flagged this directly ("make it presentable... it looks too bland"). The admin panel's CSS was rewritten in full to match the public site's brand system: Playfair Display for headings and Poppins for body text (the same Google Fonts pairing used site-wide), a black-to-burgundy gradient with a gold radial glow on the header and login screen, gold-underline navigation links, hover-lift shadows on cards, and gold-accented form focus states. Verification was done visually via Playwright screenshots across the login screen, dashboard, a content list, an edit form, and the bookings list with real data. This pass also surfaced a real (non-cosmetic) bug: the seeded "Wedding Ushering" service used the Font Awesome class `fa-rings-wedding`, which does not exist in the Font Awesome Free icon set (confirmed by inspecting the Font Awesome CDN's own CSS) — it was silently rendering as a blank circle on both the admin panel and the public Services page. It was corrected to the real class, `fa-ring`, in the seed data, the model's documentation comment, the admin form's hint text, and the already-seeded local database row.

Phase 9 — Choosing and Wiring Postgres

Local development had been running on SQLite the entire time — perfectly fine for a single developer iterating locally, but unsuitable for production, since Railway's filesystem is ephemeral and a SQLite file living on it would be silently wiped on every redeploy. The developer asked for a database recommendation; **Railway's own managed Postgres plugin** was recommended specifically because the codebase's database layer (`app/database.py`) already auto-detects and adapts a Postgres connection string with zero code changes needed, and because it keeps all moving parts (app + database) inside the same platform for a solo developer, rather than juggling a second external service. The developer added the plugin from the Railway dashboard.

Before trusting this, the connection layer was hardened: `pool_pre_ping` was added to the SQLAlchemy engine specifically for Postgres connections (not SQLite), because managed Postgres hosts silently close connections that sit idle, and without pre-ping the next request to reuse one of those dead connections fails with a confusing "SSL connection has been closed unexpectedly" error rather than transparently reconnecting. This was verified by spinning up a real local Postgres instance (via the Homebrew-installed `postgresql@18` already on the developer's machine), pointing the app at it, and running a full round-trip test: table creation, seed content insertion, every public page loading correctly, a booking submitted through the public API, and that booking appearing correctly in the admin panel after logging in. The test database was torn down afterward and the change was committed and pushed.

Phase 10 — Discovering and Fixing Production Was Not Actually Using Postgres

Testing the live Railway deployment (once the developer shared the live URL) revealed that adding the Postgres plugin to a Railway project does **not** automatically wire it to another service — the web service had no `DATABASE_URL` variable at all, meaning it was still silently running on ephemeral SQLite in production despite Postgres being present in the project. Digging further (via the Railway CLI, once installed and authenticated) revealed the same gap applied to every other secret: `SECRET_KEY`, `ADMIN_USERNAME`, and `ADMIN_PASSWORD` were all still unset, meaning production was running on the code's hardcoded defaults — critically, a `SECRET_KEY` of "change-me", which is not actually secret at all since it's the literal default value sitting in the public GitHub repository's `.env.example` file, and an admin password of "change-me".

Before making any changes, this was checked for actual exploitability: a request was sent to the live URL with a spoofed `Host` header set to `admin.localhost`, to see whether the admin panel (which is matched by `Host` header — see Phase 4) could be reached by anyone who simply forged that header. It returned a 404, confirming Railway's edge does not forward an arbitrary client-supplied `Host` header through to the app for its own routing — so this was not an active, exploitable vulnerability, but it did mean the admin panel was completely unreachable in production, by design or otherwise, until a real subdomain is configured (see Section 9).

Three production variables were then set via the Railway CLI: `DATABASE_URL` (set as a live reference to the Postgres plugin's own connection string, `${Postgres.DATABASE_URL}`, rather than a copy-pasted value, so it always tracks the plugin), a freshly generated random `SECRET_KEY`, and a new `ADMIN_PASSWORD` (a placeholder value chosen by the developer for testing, with an explicit intent to rotate it before real client use). This triggered an automatic redeploy. Afterward, the deployment was re-verified end to end: all public pages still returned success, the seeded content was intact and correctly re-created in the new Postgres database, and a fresh test booking submitted through the public API returned success — confirming writes now land in real, persistent Postgres rather than a filesystem that gets wiped on every deploy.

Phase 11 — Current State

The application is fully built, fully tested, and live on Railway with a working Postgres-backed database and rotated core secrets. The business owner cannot yet use the admin panel in production, because the client has not yet decided on a domain name, and the admin panel's virtual-host routing depends on a real subdomain with matching DNS. The developer is deliberately holding off on finalizing the go-live domain configuration until that decision is made, and continues to use the site locally (SQLite, `admin.localhost`) for any further development in the meantime.

3. Stack

LAYER	CHOICE	WHY
Language / framework	Python 3 + FastAPI	Chosen mid-project, replacing Node/Eleventy — the maintainer is Python/FastAPI-fluent, not JS
Templating	Jinja2 (via Starlette's <code>Jinja2Templates</code>)	Ships with FastAPI; real template inheritance (<code>{% extends %}</code> / <code>{% block %}</code>)
Content storage	Database rows (SQLAlchemy models), edited through the admin panel	No git/GitHub literacy required for the business owner to manage content
Database	SQLAlchemy 2.x ORM — SQLite locally, Postgres in production (Railway plugin)	Bookings plus all editable content live in the same database; Postgres survives redeploy, SQLite doesn't
Auth (whole admin panel)	Single hardcoded admin user (env vars) + signed cookie session (<code>itsdangerous</code>)	One internal user (the business owner) — no need for a user table
Server	uvicorn (ASGI)	Standard FastAPI runtime
Client-side JS	Vanilla files under <code>app/static/js/</code> — <code>main.js</code> plus <code>location-picker.js</code>	Mobile nav, theme toggle, FAQ accordion, gallery filter/lightbox, booking form fetch, map picker — no framework, no build step
Maps	Leaflet + OpenStreetMap tiles + Nominatim reverse geocoding (all free, no API key)	Avoids requiring the client to set up a billed Google Cloud project just for a map
Hosting	Railway — one web service + one Postgres plugin service, same project	Single platform for a solo developer; git-push deploys straight from GitHub

There is no Node.js, npm, or build step anywhere in this project. Everything — every public page, the booking API, and the admin panel — is one FastAPI application, run with `uvicorn app.main:app`.

One admin panel, served as a separate virtual host

Bookings and all content management (services, gallery, testimonials, FAQ, site settings) live behind a single login in `app/admin/` — a self-contained FastAPI app with its own top-level routes (`/`, `/login`, `/bookings`, `/services`, `/gallery`, `/testimonials`, `/faq`, `/settings`). It is mounted onto the main app via Starlette's `Host()` matcher (see `app/main.py`), keyed to `ADMIN_HOSTNAME` — `admin.localhost` locally (resolves to `127.0.0.1` in any modern browser, no `/etc/hosts` editing needed), a real subdomain (e.g. `admin.gpsusheringandevents.com`) once the client's domain is chosen. Because it's a distinct virtual host rather than a path prefix, every admin-side link and redirect is a plain top-level path that resolves correctly on whichever hostname it's reached through — no path-rewriting middleware needed anywhere.

There used to be a second, separate content-editing surface here (Decap CMS, a git-commit-based CMS, requiring a GitHub OAuth app). It was removed in favor of the single database-backed admin panel above — see Phase 3.

4. Project Structure

```
.
├── app/
│   ├── main.py          # Public FastAPI app: static mounts, routers, DB init, Host() mount for admin
│   ├── config.py        # pydantic-settings — reads env vars / .env
│   ├── database.py      # SQLAlchemy engine/session (sqlite:// or postgresql+psycopg://), pool_pre_ping for Postgres
│   ├── models.py        # Booking, Service, Testimonial, GalleryItem, FAQItem, SiteSetting
│   ├── schemas.py       # Pydantic request/response schemas (bookings API)
│   ├── security.py      # Signed-cookie session helpers, shared by the whole admin panel
│   ├── content.py       # Reads the DB tables above -> plain dicts for the public templates
│   ├── seed.py          # One-time seed of the original starter content into an empty DB
│   └── routers/
│       ├── pages.py     # Every public page (/ , /about.html, ... /book-us.html)
│       └── bookings.py  # POST /api/bookings (public, honeypot-filtered)
├── admin/               # The whole admin panel, its own FastAPI app (see Section 3)
│   ├── __init__.py     # Builds admin_app, mounts its static/images, includes routers
│   └── routers/
│       ├── auth.py      # /login, /logout
│       ├── home.py      # / — dashboard landing with counts per section
│       ├── bookings.py  # /bookings — list + status updates
│       ├── services.py  # /services — full CRUD
│       ├── gallery.py   # /gallery — full CRUD + image upload
│       ├── testimonials.py # /testimonials — full CRUD
│       ├── faq.py       # /faq — full CRUD
│       └── settings.py  # /settings — single-row edit form
├── templates/
│   ├── base.html        # Public layout: head, topbar, navbar, footer, WhatsApp float
│   ├── partials/        # topbar.html, navbar.html, footer.html
│   ├── pages/           # index.html, about.html, services.html, ...
│   └── admin/           # login, home, bookings, and each content type's list/form templates
├── static/
│   ├── css/style.css    # All public-site styling — CSS vars, light/dark theme tokens
│   ├── js/main.js       # All public-site client-side interactivity
│   ├── js/location-picker.js # Book Us page's Leaflet map picker
│   └── admin.css        # Admin-panel-only styling — brand-matched (Playfair/Poppins, gold/black/burgundy)
├── images/uploads/      # Photos uploaded through the Gallery admin section land here
├── docs/                # Non-technical project docs (this file's siblings)
├── requirements.txt
├── Procfile             # `web: uvicorn app.main:app --host 0.0.0.0 --port $PORT`
└── .env.example
```

5. How a Page Request Actually Resolves

1. A route in `app/routers/pages.py` (e.g. `GET /services.html`) calls `content.get_services()`, which opens a short-lived DB session, queries the `services` table ordered by its `order` column, converts each row's `highlights` text blob into a list (split on newlines), and returns plain dicts.
2. The route builds a context dict via a shared `base_context(request, **extra)` helper (also in `pages.py`), which always injects `settings` (from the `site_settings` table) and `footer_services` (first 4 services, for the footer's link list).
3. `templates.TemplateResponse(request, "pages/services.html", context)` renders that template, which does `{% extends "base.html" %}` and fills in `{% block content %}`.
4. `base.html` pulls in the shared `partials/topbar.html`, `navbar.html`, `footer.html`, all reading from the same `settings` context variable.

Nothing is pre-built or cached — every request re-reads the database fresh. An admin panel save (add a testimonial, upload a gallery photo, edit a FAQ answer) is live on the very next public-page request, no rebuild step, no cache to bust. `app/seed.py` runs once at startup and only inserts the original starter content if a table is completely empty, so it never overwrites anything the business owner has since added or changed.

Admin panel auth

Single admin user, credentials from `ADMIN_USERNAME/ADMIN_PASSWORD` env vars, compared with `hmac.compare_digest` (timing-safe). On success, `app/security.py` issues an `itsdangerous`-signed cookie (no server-side session store — the cookie itself is the session, verified by signature plus a 12-hour max age on every protected request). The same `require_admin` dependency guards every route under `app/admin/` — bookings and all content management alike. `COOKIE_SECURE` must be `false` for local `http://` testing (browsers silently drop a Secure cookie over plain HTTP, which is exactly the kind of bug that looks fine in curl but breaks in a real browser) and must stay `true` in production.

The Book Us map picker

`app/static/js/location-picker.js` initializes a Leaflet map in `#location-picker` centered on Accra. Clicking the map or dragging its marker reverse-geocodes the picked coordinates via OpenStreetMap's free Nominatim API and writes the resulting address into the `#location` input — typing the address directly still works exactly as before, this is purely an assist. No API key, no billing account, and nothing server-side to configure.

Nominatim's usage policy asks for an identifying User-Agent or Referer for heavier automated use; browsers can't set a custom User-Agent from JS, but the page's Referer satisfies it for this kind of light, occasional client-side traffic. A site seeing heavy booking-form volume should move this behind a server-side proxy or a paid geocoding provider instead.

6. Data Model

TABLE	HOLDS
bookings	name, phone, email, event_type, event_date, guest_count, location, message, a status enum (new/contacted/confirmed/completed/cancelled), created_at
services	title, icon (Font Awesome class), order, home_description, description, highlights (newline-separated text)
testimonials	name, role, rating, order, quote
gallery_items	label, category, order, image (path under /images/uploads/, or empty)
faq_items	question, order, answer
site_settings	Single row (id=1) — site_name, site_url, tagline, phone_display, whatsapp_number, email, address, facebook/instagram/tiktok URLs

POST /api/bookings (app/routers/bookings.py) is public and unauthenticated by design — it's the public booking form's submit endpoint — but silently drops anything where the bot_field honeypot is filled in, without storing or acknowledging it any differently to a bot. Every other table is only ever written to through the admin panel.

app/database.py rewrites a Railway-style postgres:// URL to postgresql+psycopg:// (psycopg 3) automatically, and adds pool_pre_ping for any non-SQLite connection; locally it defaults to sqlite:///./bookings.db with no setup required.

7. Running It Locally

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
ADMIN_USERNAME=admin ADMIN_PASSWORD=devpass SECRET_KEY=dev COOKIE_SECURE=false \
  uvicorn app.main:app --reload --port 8000
```

Then `http://localhost:8000/` for the public site, and `http://admin.localhost:8000/` for the admin panel (login: admin/devpass) — bookings, services, gallery, testimonials, FAQ and settings all live there behind that one login.

8. Troubleshooting Log — Every Bug Hit, and the Actual Fix

Kept in full because the diagnostic path is often more useful later than the fix itself.

SYMPTOM	ROOT CAUSE	FIX
Eleventy build crawled into <code>node_modules</code> / a Python venv	No ignore rules configured	<code>.eleventyignore</code> (moot — Eleventy later removed entirely)
Settings data file silently not loading	Eleventy doesn't parse YAML data files without extra config	Converted to JSON (moot)
Markdown content files rendering as stray standalone pages	Eleventy's default collection behavior	Per-directory <code>permalink: false</code> control files (moot)
Existing links broke after a build	Eleventy's default pretty-URL output	Explicit <code>permalink: front matter</code> (moot)
GitHub Dependabot flagged 13 vulnerabilities	Pinned dependency versions had gone stale	Bumped every dependency to current releases
<code>TypeError: cannot use 'tuple' as a dict key</code> on every page	The dependency bump pulled in Starlette 1.6.0, which changed <code>TemplateResponse</code> 's calling convention; old code passed <code>(name, context)</code> as a tuple	Added <code>request</code> as the required first positional argument to all 7 calls in <code>pages.py</code>
Stray rebuilt <code>_site/</code> kept reappearing after Eleventy was deleted	A leftover background <code>eleventy --serve</code> process was still running	Found and killed the specific PIDs
<code>psycpg2-binary</code> failed to build	No prebuilt wheel for Python 3.14	Switched to <code>psycpg[binary]</code> v3, updated the DB URL scheme to <code>postgresql+psycpg://</code>
SQLAlchemy crashed on <code>Mapped[str None]</code> annotations	SQLAlchemy 2.0.35 incompatibility with Python 3.14	Bumped to SQLAlchemy 2.0.52
<code>pydantic.EmailStr</code> raised an import error	Missing optional dependency	Added <code>email-validator</code> to requirements
Admin login appeared to silently fail in a real browser, but worked fine in curl tests	Real browsers reject a <code>Secure</code> cookie over plain <code>http://</code> ; curl doesn't enforce this, masking the bug	Added a <code>COOKIE_SECURE</code> setting, <code>false</code> for local dev, <code>true</code> in production
Flaky curl test results (wrong counts, wrong routing)	Shell word-splitting mangling <code>-H "Host: ..."</code> headers in test scripts	Not an app bug — resolved by always writing curl headers fully literally
"Wedding Ushering" service icon rendered as a blank circle, on both admin and the public Services page	<code>fa-rings-wedding</code> is not a real Font Awesome Free icon class — only <code>fa-ring</code> exists	Fixed in seed data, model doc comment, admin form hint text, and the already-seeded database row

Production silently still on SQLite despite the Postgres plugin existing in the Railway project	Adding a database plugin to a Railway project does not automatically wire its connection string into other services — <code>DATABASE_URL</code> must be explicitly set/referenced on the actual web service	Set <code>DATABASE_URL=\${{Postgres.DATABASE_URL}}</code> via the Railway CLI on the <code>web</code> service specifically, then redeployed
Production admin login and cookie-signing secret were both still on their public, hardcoded default values	No env vars had ever been set on the Railway web service beyond what Railway injects automatically	Generated and set a real random <code>SECRET_KEY</code> and a new <code>ADMIN_PASSWORD</code> via the Railway CLI

9. Deployment & Infrastructure — Current State

ITEM	CURRENT STATE
Host	Railway, project name <code>adaptable-truth</code> , region <code>sfo</code>
Services in the project	<code>web</code> (the FastAPI app) and <code>Postgres</code> (managed database plugin, volume <code>postgres-volume</code>)
Live URL	A temporary Railway-assigned <code>*.up.railway.app</code> address — functional now, will move to the client's real domain once chosen
Deploy trigger	Push to <code>master</code> on <code>Kofi-Astro/Ushering</code> — Railway is linked to the GitHub repo and redeploys automatically
DATABASE_URL	Set, referencing the Postgres plugin's connection string directly (<code>{{Postgres.DATABASE_URL}}</code>) rather than a copied value, so it always tracks the plugin
SECRET_KEY	Set to a freshly generated random value — no longer the public repo default
ADMIN_USERNAME / ADMIN_PASSWORD	Username still the default (<code>admin</code>); password was set to a placeholder for testing at the developer's request and still needs to be rotated to a real password before the business owner is given access
ADMIN_HOSTNAME	Not yet set — still defaults to <code>admin.localhost</code> , which is unreachable from the public internet. The admin panel is therefore not usable in production yet (see Section 10)
Gallery uploads (<code>images/uploads/</code>)	Still written to local disk — not yet moved behind a persistent Railway volume or object storage, so uploaded photos will not survive a redeploy until this is addressed
Railway CLI	Installed locally via Homebrew, authenticated via GitHub OAuth (<code>railway login</code>), linked to the <code>web</code> service
Direct Postgres access	Not yet set up — <code>railway connect Postgres</code> requires a registered SSH key on the account (<code>railway ssh keys add</code>), and no public TCP proxy is currently exposed for the database (private-networking only, which is the more secure default)

A spoofed Host: `admin.localhost` header was sent to the live production URL to check whether the admin panel could be reached by anyone forging that header. It returned a 404 — Railway's edge does not forward an arbitrary client-supplied Host header through to the app, so this is not an active vulnerability. It does mean the admin panel simply cannot be used at all until a real subdomain is configured.

Account credentials should never be pasted directly into a chat with an AI assistant or any other tool — during this project a Railway/Google account password was shared this way, and the recommendation given at the time (and repeated here) was to rotate it immediately and use `railway login`'s browser-based OAuth flow instead, which never requires typing a password into a terminal or a third party.

10. Remaining Work — Pre-Launch Checklist

1. **Domain decision (blocking).** Waiting on the client. Once chosen: register it, point its DNS at Railway (root/www → the site, an admin. subdomain → the admin panel), add it as a custom domain in the Railway dashboard, then set `ADMIN_HOSTNAME` to match.
2. **Rotate the admin password.** The current production `ADMIN_PASSWORD` is a testing placeholder chosen by the developer — replace it with a real, strong password before handing access to the business owner, and consider changing `ADMIN_USERNAME` away from the default `admin` at the same time.
3. **Persistent storage for gallery uploads.** Attach a Railway volume mounted at `images/uploads/`, or move uploads to object storage (e.g. Cloudflare R2), before real client photos are uploaded — otherwise they're lost on the next redeploy exactly like the SQLite data was.
4. **Populate real business content.** Real contact details, logo, and photography from the client still need to replace the seeded placeholder content across services, gallery, and site settings.
5. **Basic rate limiting** on `POST /api/bookings` beyond the honeypot field, once the site is getting real public traffic.
6. **Automated tests.** Nothing beyond manual `curl`/Playwright verification exists yet — worth adding at least smoke-level coverage of the booking flow and admin CRUD before this scales into a template for future projects.
7. **Pagination** on the bookings list and content lists — a non-issue at current volumes, will matter as bookings accumulate.

11. Playbook — What to Reuse on the Next Project

The GPS Ushering build produced a set of decisions and hard-won lessons that generalize well beyond this one client. Kept here deliberately as standalone, reusable guidance.

- **Match the stack to the maintainer's actual fluency, not to what's fashionable.** A stack the solo developer doesn't know creates permanent drag — every bug takes longer, every feature needs research. Ask this question before writing a line of code, not after the first prototype is already built.
- **For a non-technical client who needs to self-manage content, avoid git-based CMS tools entirely** (Decap CMS, Netlify CMS, and similar). They require understanding commits and a GitHub account, which most business owners simply don't have and shouldn't need. Model content as database rows edited through plain CRUD forms instead.
- **Serve an admin interface on its own subdomain via host-based routing, not a path prefix.** A separate virtual host (Starlette's `Host()`, or the equivalent in another framework) means every internal link and redirect in the admin app is a plain top-level path — no prefix-rewriting logic needed anywhere, and the admin app can be developed as if it were a completely standalone application.
- **Skip building a user table for a single-operator admin panel.** One hardcoded account (env-var credentials, signed-cookie session, no server-side session store) is simpler to build and reason about than a full auth system, and is entirely appropriate when there's genuinely only one person who will ever log in.
- **Avoid pushing paid, billing-gated API setups onto non-technical clients** (the Google Maps JavaScript API being the canonical example). Free, no-key alternatives (Leaflet + OpenStreetMap + Nominatim) cover the common case — an address picker, a static embed — without requiring the client to ever touch a cloud billing console.
- **Always add connection pre-pinging for managed Postgres.** Idle connections get silently dropped by the host; without it, the failure mode is a confusing intermittent SSL error under real traffic rather than a clean, transparent reconnect.
- **Never assume a managed platform auto-wires a newly added plugin to an existing service.** Adding a database plugin on Railway (or the equivalent on any PaaS) does not automatically inject its connection string anywhere else — explicitly check the actual running service's environment variables, don't just check that the plugin exists in the project.
- **Before calling anything "production-ready," explicitly audit every secret for its literal default value.** A `SECRET_KEY` or password left at whatever the code ships with by default is not actually secret if that default is visible in a public repository's `.env.example` — this is easy to overlook because the app still works perfectly with defaults in place, right up until someone exploits them.
- **Commit at every meaningful stage, including mid-project pivots.** A clean, sequential git history is both a safety net (easy to revert a bad direction) and a legible record of how and why the architecture evolved — invaluable when picking a project back up months later.
- **Produce separate documents for separate audiences.** A client-facing project overview, a client-facing pricing brief, and a developer-facing technical overview serve entirely different readers and shouldn't be collapsed into one file — what a client needs to know before a project starts is not what a developer needs to know to maintain the code afterward.
- **When pricing is genuinely uncertain, produce multiple variants rather than guessing.** A blank-priced and a fully-priced version of the same proposal is cheap to produce and lets the developer choose the right one for each client conversation, without a slow back-and-forth mid-negotiation.
- **Never type or paste real account credentials into a terminal, chat, or third-party tool.** Every legitimate CLI (Railway, GitHub, etc.) supports browser-based OAuth login for exactly this reason — use it, and rotate any password that ever gets typed somewhere it shouldn't have been.